5. IMPLEMENTATION AND TESTING

5.1 Implementation Approach

5.1.1 Introduction

Module 5 implements the interactive answer and evaluation stage of the AI Interview Preparation Platform. After Module 4 creates role-specific questions, the candidate can type an answer or dictate it using the microphone. Each response is validated, associated with the authenticated user, stored in MongoDB Atlas, evaluated through Gemini AI or a permanent local fallback, and presented with scores and improvement feedback.

The implementation follows the existing MVC architecture. React manages the interview interface; Axios sends authenticated requests; Express routes requests to controllers; service modules perform AI or rule-based evaluation; Mongoose validates and stores answers; and the final report aggregates all question-level evaluations.

- Typed and voice-to-text answer input with editable transcripts.
- User, interview and question ownership validation through JWT-protected APIs.
- Five evaluation scores: correctness, confidence, communication, technical accuracy and overall performance.
- Strengths, weaknesses and suggestions for every submitted answer.
- Gemini evaluation with a deterministic local fallback when the external service is unavailable.
- Interview completion validation and a persistent final performance report.

5.1.2 Input and Output Design Implementation

The main input is the candidate's response to the displayed interview question. The answer may be entered directly in a textarea or generated as text through browser speech recognition. The interface limits each response to 5,000 characters, displays a live counter, prevents blank submissions and disables navigation while voice recognition or saving is in progress.

Input / Output	Implementation	Validation or Purpose
Typed answer	React controlled textarea	2-5,000 characters; editable before submission
Voice answer	Web Speech API transcript	Microphone permission, browser support and recognition errors handled
Answer request	questionId and answer text	JWT identifies the user; URL identifies the interview
Evaluation output	Scores and feedback lists	Each score is constrained between 0 and 100
Final report	Interview averages and detailed review	Available only after every question is evaluated

5.1.3 Database Implementation

MongoDB Atlas provides permanent, user-specific storage. The Answer model references the User, Interview and Question collections. A compound unique index on userId, interviewId and questionId ensures that one candidate has only one current response per interview question. Re-submission updates the same document and resets the previous evaluation before calculating new feedback.

The Interview model stores the final status, overall score, completion time and a structured performanceReport object. This report contains category averages and combined strengths, weaknesses and suggestions, allowing Module 6 to generate history and analytics without recalculating completed interviews.

5.1.4 Collection Structures

Collection	Important fields	Relationship / Role
Users	_id, name, email, password	Owns resumes, interviews and answers
Resumes	userId, analysis, interviewPreferences	Supplies candidate context and target role
Interviews	userId, resumeId, role, score, performanceReport	Stores configuration and final result
Questions	interviewId, order, question, expectedPoints	Provides the evaluation rubric
Answers	userId, interviewId, questionId, answer, scores, feedback	Stores each evaluated response

5.1.5 Code Modules

Layer	File	Responsibility
Model	server/models/Answer.js	Answer constraints, evaluation fields and unique index
Controller	server/controllers/answerController.js	Save, evaluate, retrieve and complete interviews
Route	server/routes/answerRoutes.js	Protected answer and completion endpoints
AI service	server/services/geminiAnswerEvaluator.js	Structured contextual evaluation through Gemini
Fallback	server/services/localAnswerEvaluator.js	Rule-based evaluation available without Gemini
Orchestration	server/services/answerEvaluationService.js	Gemini attempts and automatic fallback selection
Frontend	client/src/pages/MockInterview.jsx	Text input, microphone input, saving and feedback display
Frontend	client/src/pages/InterviewReport.jsx	Final category scores and detailed answer review

5.1.6 System Implementation

When the candidate selects Save and Continue, the frontend sends POST /api/answers/:interviewId. The controller validates MongoDB identifiers, checks that the interview belongs to the authenticated user, verifies that the question belongs to that interview and saves the normalized answer. It then invokes the evaluation service and stores the result before returning it to the interface.

On the final question, POST /api/answers/:interviewId/complete verifies that the number of completed evaluations equals the number of interview questions. It calculates category averages, creates the combined report, changes the status to completed and navigates the candidate to /interview/:interviewId/report. Opening this route after a refresh reloads the persisted interview and answers.

5.1.7 Project Summary

Module 5 converts the question-display prototype into a complete mock-interview experience. It supports accessible answer entry, secure persistence, resilient evaluation and

actionable reporting. The implementation remains modular so the AI provider, scoring rules or input method can be changed without rewriting the entire application.

5.2 Coding Details and Code Efficiency

Representative code from the implemented module is shown below. Only focused excerpts are included so that the report explains the important logic without duplicating entire source files.

A. Answer ownership and duplicate prevention - Answer.js

```
answerSchema.index(
  { userId: 1, interviewId: 1, questionId: 1 },
  { unique: true }
);

answer: {
  type: String,
  required: true,
  trim: true,
  minlength: 2,
  maxlength: 5000,
}
```

The compound index supports fast ownership queries and prevents duplicate responses. Mongoose validation rejects invalid text before it reaches permanent storage.

B. Browser voice-to-text initialization - MockInterview.jsx

```
const SpeechRecognition =
  window.SpeechRecognition || window.webkitSpeechRecognition;

const recognition = new SpeechRecognition();
recognition.lang = navigator.language || "en-US";
recognition.continuous = true;
recognition.interimResults = true;
recognition.start();
```

The browser converts speech into editable text. The platform stores only the transcript, not an audio file, reducing database storage and privacy exposure. Unsupported-browser, permission, microphone and recognition-service errors are displayed safely.

C. Gemini with permanent local fallback - answerEvaluationService.js

```
for (const model of GEMINI_MODELS) {
  try {
    const evaluation = await evaluateWithGemini({ ...options, model });
    return { ...evaluation, source: "gemini", model };
  } catch (error) {
    lastError = error;
  }
}

return { ...evaluateLocally(options), source: "local", model: null };
```

The orchestration service isolates external API failure from the controller. Gemini results are schema-validated; if all attempts fail, the local evaluator returns an identical response structure so the UI continues to work.

D. Weighted local overall score - localAnswerEvaluator.js

```
const overall = clamp(
  correctness * 0.35 +
  confidence * 0.20 +
  communication * 0.25 +
  technicalAccuracy * 0.20
);
```

Correctness receives the largest weight, while communication, confidence and role-specific accuracy also contribute. The clamp function guarantees an integer score between 0 and 100.

E. Final report aggregation - answerController.js

```
const average = (field) => Math.round(
  answers.reduce((total, item) => total + item.scores[field], 0)
  / answers.length
);

interview.status = "completed";
interview.score = average("overall");
interview.completedAt = interview.completedAt || new Date();
await interview.save();
```

5.2.1 Code Efficiency Measures

- MVC separation keeps database, request-handling, evaluation and interface responsibilities independent.
- Database indexes improve common lookups and enforce integrity without repeated application-side scans.
- findOneAndUpdate with upsert updates an existing answer or creates it in one database operation.
- Expected answer points stored with each question avoid regenerating an evaluation rubric.
- Structured Gemini JSON validation prevents malformed AI data from entering MongoDB.
- Bounded feedback arrays and answer length limits prevent uncontrolled document growth.
- The local evaluator uses sets for unique tokens and direct keyword lookup, keeping processing proportional to answer length.
- React stores answers by question ID, enabling constant-time retrieval when navigating between questions.

5.3 Testing Approach

Testing was performed at unit, integration and complete-system levels. Automated syntax checks, module import checks, ESLint and Vite production builds were combined with browser tests for answer entry, voice transcription, persistence, evaluation and report navigation. Test data used a five-question interview to keep repeated end-to-end checks efficient.

5.3.1 Unit Testing

Unit-level checks focused on isolated validation and scoring logic. Node's syntax checker verified every new backend module. The local evaluator was executed with a

controlled technical question and answer, and the test confirmed that it returned five integer scores, strengths, weaknesses and suggestions within the required bounds.

Unit	Test performed	Expected result	Result
Answer validation	Submit fewer than 2 or more than 5,000 characters	HTTP 400 with clear message	Pass
ObjectId validation	Invalid interview or question ID	Request rejected before query	Pass
Local scoring	Evaluate controlled answer text	Scores are integers from 0 to 100	Pass
Gemini validation	Malformed or incomplete response	Reject response and use fallback	Pass
Frontend code	Run ESLint	No lint errors	Pass

5.3.2 Integration Testing

Integration testing verified communication between React, Axios, JWT middleware, controllers, services and MongoDB Atlas. A saved answer was revisited and remained visible after navigation and refresh. Changing and resubmitting it updated the same database record. The evaluation panel displayed the source and persisted feedback returned by the protected API.

Integration	Components	Observed result
Authentication	Axios + JWT middleware + User model	Only authenticated users access answer APIs
Answer persistence	MockInterview + controller + Answer model	Answer survives navigation and refresh
Evaluation	Controller + Gemini/local services + MongoDB	Scores and feedback saved with source
Completion	Questions + Answers + Interview model	Incomplete interview rejected; complete report stored
Report reload	React Router + interview/answer APIs	Completed report opens after refresh

5.3.3 System Testing

System testing followed the complete user workflow: register, log in, upload a resume, generate an interview, answer using text and microphone, inspect individual feedback, complete all questions and open the final report. The backend remained connected to MongoDB Atlas, and the application displayed friendly empty, loading and error states rather than exposing technical database details.

System test	Acceptance criterion	Result
Typed-answer workflow	Save, navigate back and reload the stored response	Pass
Voice-to-text workflow	Start, stop, edit and submit transcript in Chrome	Pass
Fallback availability	Local questions/evaluation returned when Gemini unavailable	Pass
Final performance report	All category averages and detailed review displayed	Pass
Production frontend	Vite production build completes	Pass
Backend structure	Syntax and import checks complete	Pass

5.4 Modifications and Improvements

5.4.1 Modifications Completed

- Replaced the original practice-note placeholder with a real controlled answer field.
- Changed the technical user message from 'Saved in MongoDB' to the clearer 'Answer saved'.
- Added browser voice-to-text with start/stop controls, interim transcripts and permission errors.
- Added ownership checks so users cannot read or submit answers for another user's interview.
- Added answer upsert behavior to prevent duplicate documents when an answer is edited.
- Added Gemini evaluation and a permanent local fallback with a visible source label.
- Added final-interview validation, aggregate scoring and a refresh-safe performance-report route.
- Added responsive score cards and feedback panels for desktop and mobile screens.

5.4.2 Current Limitations

- Web Speech API support and recognition quality vary by browser, language and external speech service availability.
- Only the transcript is analysed; actual voice tone, pauses, facial expression and body language are not measured.
- The local evaluator is deterministic and less context-aware than Gemini.
- Gemini availability remains subject to network access, model availability and API quota.
- Keyword-based resume parsing may miss unusual resume layouts or uncommon terminology.

5.4.3 Future Improvements

- Add server-side multilingual speech transcription for consistent browser support.
- Add optional audio/video recording with explicit consent and secure retention controls.
- Measure delivery pace, filler words and pauses when audio analysis is available.
- Add coding questions with an editor and automated test cases.
- Use Module 6 history to generate personalized practice plans from repeated weaknesses.
- Add automated backend API tests with a dedicated test database and continuous integration.

5.4.4 Conclusion

Module 5 successfully implements the core interview-answering and evaluation workflow. Its secure MongoDB relationships, hybrid evaluation strategy and persisted performance report provide a reliable foundation for interview history and analytics. The module meets the functional requirement of returning scores, strengths, weaknesses and practical suggestions for every answer while remaining operational during external AI failure.